

Software engineering fundamentals that work with humans also work exceptionally well with AI—treat AI as a collaborative partner, not a code generator.

[

LLM Constraints: Smart Zone vs Dumb Zone

- LLMs have a ‘smart zone’ (first 100K tokens) where they perform best, then degrade into a ‘dumb zone’ as context grows—attention relationships scale quadratically like teams in a league
- Size tasks to fit the smart zone; avoid long sessions that accumulate context debt
- Compacting (summarizing history) creates sediment and inconsistency—prefer clearing context and starting fresh like ‘the guy from Memento’
- Every session follows: system prompt → exploration → implementation → testing. Clearing resets to system prompt, ensuring consistent starting state

Keep your system prompt tiny. 250K tokens in the system prompt puts you straight into the dumb zone without doing anything.

PRD: Destination Document

- After grilling, write a Product Requirements Document (PRD) summarizing the design concept—problem statement, solution, user stories, implementation decisions, testing decisions
- Don’t review the PRD obsessively—you’ve already aligned with the AI, and LLMs excel at summarization
- PRD defines ‘where we’re going’ and out-of-scope items (definition of done)
- Store PRDs as GitHub issues or local markdown files for traceability

AFK Implementation: Night Shift for AI

- Human handles ‘day shift’ (planning, grilling, PRD, kanban)—AI handles ‘night shift’ (AFK implementation)
- Ralph loop: AI picks next task from backlog (prioritizing bugs, infra, tracer bullets, polish), explores repo, uses TDD, runs feedback loops
- Pass all issues + last 5 commits into context; AI outputs ‘no more tasks’ when done
- Run in Docker sandbox for safety; use ‘once’ script for human-in-the-loop tuning before full AFK loop
- Token budget awareness essential: display exact token count in status line to monitor proximity to dumb zone

],

[

Grill Me Skill: Alignment Over Specs

- ‘Specs to code’ fails because you lose touch with the codebase—code is your battleground, not an afterthought
- Use ‘grill me’ skill to reach shared understanding (design concept) with AI through relentless questioning before writing anything
- AI provides recommended answers to its own questions, speeding alignment while keeping you in the loop
- This is human-in-the-loop work (not AFK)—can involve 40-100 questions, creating conversation history as a design asset
- Pair/mob programming with AI: domain expert + developer + AI in same room for crucial decisions

I needed to reach a shared understanding with Claude. I didn’t need an asset, I didn’t need a plan—I needed to be on the same wavelength.

Kanban Board: Vertical Slices Over Horizontal Layers

- Break PRD into independently grabbable issues using vertical slices (tracer bullets)—thin slices crossing all layers (DB, API, frontend) for immediate feedback
- AI defaults to horizontal coding (phase 1: all DB, phase 2: all API, phase 3: frontend)—this delays feedback until phase 3
- Vertical slices enable parallelization: multiple agents can work simultaneously on non-blocking tasks
- First slice should be minimal but complete: schema changes + service + visible frontend representation
- Kanban structure creates directed acyclic graph (DAG) for optimal agent scheduling

Even though I told it to do vertical slices, it proposed creating the gamification service first on its own. That’s a horizontal slice. I want schema + service + minimal frontend in the first slice.

Code Review & Quality

- Bad codebases produce bad agents—deeply understanding TypeScript, code structure, and tooling makes you a better AI-assisted developer
- More code review is inevitable when delegating implementation to AI—no way around it yet
- Read code before making claims; investigate before answering; verify builds and tests after changes
- Use sub-agents for broad exploration (isolated context, summary returned); use direct tools for specific lookups
- Inversion of control: own your planning stack rather than relying on opaque frameworks—when things break, you need to know how to fix them

If you have a garbage codebase, you’re going to get garbage out of the agent working in that codebase.

]